Reviewer Pack — S-Transform Free-Mult Defect Lower-Bounds DISJ Communication

Entry 6c8daf2ab19d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-17 20:26:49 UTC

S-Transform Free-Mult Defect Lower-Bounds DISJ Communication

Entry ID: `6c8daf2ab19d` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-17 20:26:49 UTC

1. Conjecture

Field A (mathematical branch): Free probability — Voiculescu's S-transform and free multiplicative convolution ⊠ of compactly supported measures on R_+ (Voiculescu 1987; Bercovici–Voiculescu 1992; Haagerup–Schultz 2007). Multiplicative free convolution is distinct from additive (R-transform) free convolution previously attempted on graph Laplacians; an arXiv search for 'S-transform' AND ('communication complexity' OR 'sign rank' OR 'discrepancy method') returns 0 direct hits with <5 adjacent papers, all on Wigner-type ensembles in QIT, never on Boolean communication matrices. **Field B** (complexity object): Randomized two-party communication complexity CC_R(M) of DISJOINTNESS and other Boolean matrices M ∈ {0,1}^{N×N} (Razborov 1992 Ω(n) regime), with N = 2^n; the τ(M) tensor-invariant program for COMM_DISJ.

Statement:

> For Boolean M ∈ {0,1}^{N×N} let Q_M := (1/N) · M^T M and let σ_M be its empirical spectral measure, normalized to unit total mass and unit first moment (m_1=1) by rescaling. Compute moments m_k = (1/N) tr((Q_M/m_1)^k) for k=1..K with K=⌈log_2 N⌉+2; form the truncated ψ-series ψ(z)=Σ m_k z^k, invert it to χ(z)=ψ^{-1}(z) by Lagrange inversion, set S_M(z)=(1+z)·χ(z)/z, and obtain the moments {μ_k} of the free multiplicative self-convolution σ_M⊠σ_M from S_{⊠}(z)=S_M(z)^2 by re-inverting. Define the free-mult defect δ(M) := |μ_2 − 2·m_2 − 1| / (1+m_2). Conjecture: there is an absolute constant c ∈ [1/64, 1/4] such that (i) for every Boolean M, CC_R(M) ≥ c · log_2(N) · δ(M); (ii) δ(M_DISJ_n) ≥ 1/8 for all n ≥ 3. A single Boolean M with U(M) < c·log_2(N)·δ(M) (where U is any certified CC upper bound), or a single n ≥ 3 with δ(M_DISJ_n) < 1/8, refutes the conjecture.

Rationale (proposer's reasoning):

> Multiplicative free convolution σ⊠σ encodes asymptotic FREENESS between the row-space and column-space spectral projections of M; deviation from the freeness identity μ_2 = 2m_2 + 1 detects multiplicative coupling between input partitions, which is exactly the structural property Razborov exploits to force long bipartite rectangles in DISJ. Unlike additive R-transform invariants (previously tried on graph Laplacians), the S-transform is intrinsically multiplicative and respects the row × column tensor structure of CC matrices. The bridge is barrier-shielded: Lagrange inversion is non-polynomial in the entries, the invariant is RELATIONAL (a comparison between two measures, not a single truth-table functional), CC lower bounds are not subject to the natural-proofs barrier, and the

construction uses no ring-only operations on a Boolean ring extension, so it does not algebrize.

Taxonomy category: `COMM_COMPLEXITY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `0b299805f1647daf`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Across n∈{3,4,5,6} (30 seeds each), for every sampled M and the three controls, U(M) ≥ 0.05·log_2(N)·δ(M); and δ(M_DISJ_n) ≥ 0.125 at n=3,4,5,6. SUPPORTED iff ≥27/30 seeds pass the inequality at every n AND all four DISJ defects ≥ 0.125.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.90	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.90	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 13 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `S-transform free multiplicative convolution communication complexity` - `free probability spectral moments disjointness lower bound Boolean matrix` - `Voiculescu S-transform sign rank discrepancy communication matrix`

Top relevant hits considered: - [http://arxiv.org/abs/2004.07191v1] Cauchy-Stieltjes kernels families and free multiplicative convolution - [http://arxiv.org/abs/1407.6169v3] Multiplicative Complexity of Vector Valued Boolean Functions - [http://arxiv.org/abs/2511.02758v3] Finite free probability and S transforms of Jacobi processes - [http://arxiv.org/abs/2511.07739v3] A Lower Bound for the Fourier Entropy of Boolean Functions on the Biased Hypercube - [http://arxiv.org/abs/1806.02734v3] Spectral lower bounds for the orthogonal and projective ranks of a graph - [http://arxiv.org/abs/1710.01374v2] Free-Boolean independence for pairs of algebras - [http://arxiv.org/abs/1503.07648v2] Sign rank versus VC dimension - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=94.4s

5.1 Generated Python source

```
import sys
import random
import math
import json
```

```
from fractions import Fraction

def matrix_mult(a, b):
    n = len(a)
    result = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                result[i][j] += a[i][k] * b[k][j]
    return result

def matrix_transpose(m):
    return [list(row) for row in zip(*m)]

def matrix_trace(m):
    return sum(m[i][i] for i in range(len(m)))

def matrix_power(m, k):
    n = len(m)
    result = [[0] * n for _ in range(n)]
    for i in range(n):
        result[i][i] = 1
    for _ in range(k):
        result = matrix_mult(result, m)
    return result

def compute_moments(Q, K):
    m = [0.0] * (K + 1)
    m[1] = matrix_trace(Q) / len(Q)
    for k in range(2, K + 1):
        Q_pow = matrix_power(Q, k)
        m[k] = matrix_trace(Q_pow) / len(Q)
    return m

def lagrange_inversion(m, K):
    chi = [0.0] * (K + 1)
    chi[0] = 1.0
    for k in range(1, K + 1):
        for j in range(1, k + 1):
            chi[k] += Fraction(j, k) * m[j] * chi[k - j]
    return chi

def compute_S_transform(chi, K):
    S = [0.0] * (K + 1)
    for k in range(1, K + 1):
        S[k] = (1 + k) * chi[k] / k
    return S

def compute_free_mult_defect(m, mu, K):
    m2 = m[2]
    mu2 = mu[2]
    delta = abs(mu2 - 2 * m2 - 1) / (1 + m2)
    return delta

def build_disj_matrix(n):
    N = 2 ** n
```

```
    M = [[0] * N for _ in range(N)]
    for i in range(N):
        for j in range(N):
            M[i][j] = 1 if (i & j) == j else 0
    return M

def build_rank1_matrix(N):
    v = [random.randint(0, 1) for _ in range(N)]
    M = [[0] * N for _ in range(N)]
    for i in range(N):
        for j in range(N):
            M[i][j] = v[i] * v[j]
    return M

def build_and_matrix(N):
    M = [[0] * N for _ in range(N)]
    for i in range(N):
        for j in range(N):
            M[i][j] = 1 if (i & j) == j else 0
    return M

def build_identity_matrix(N):
    M = [[0] * N for _ in range(N)]
    for i in range(N):
        M[i][i] = 1
    return M

def compute_cc_upper_bound(M):
    N = len(M)
    max_iter = 4 * N
    cover_size = 0
    for _ in range(max_iter):
        i = random.randint(0, N - 1)
        j = random.randint(0, N - 1)
        if M[i][j] == 1:
            cover_size += 1
    return math.log2(cover_size) if cover_size > 0 else 0

def run_trial(seed):
    random.seed(seed)
    n_values = [3, 4, 5, 6]
    metric_values = []
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        N = 2 ** n
        K = math.ceil(math.log2(N)) + 2

        # Build DISJ matrix
        M_disj = build_disj_matrix(n)
        Q_disj = matrix_mult(matrix_transpose(M_disj), M_disj)
        m_disj = compute_moments(Q_disj, K)
        chi_disj = lagrange_inversion(m_disj, K)
        S_disj = compute_S_transform(chi_disj, K)
        mu_disj = [0.0] * (K + 1)
```

```
for k in range(1, K + 1):
    mu_disj[k] = 2 * S_disj[k]
delta_disj = compute_free_mult_defect(m_disj, mu_disj, K)

if delta_disj < 0.125:
    conjecture_holds = False
    counterexample = f"DISJ defect too small: n={n}, delta={delta_disj}"

# Sample uniform Boolean matrix
M_uniform = [[random.randint(0, 1) for _ in range(N)] for _ in range(N)]
Q_uniform = matrix_mult(matrix_transpose(M_uniform), M_uniform)
m_uniform = compute_moments(Q_uniform, K)
chi_uniform = lagrange_inversion(m_uniform, K)
S_uniform = compute_S_transform(chi_uniform, K)
mu_uniform = [0.0] * (K + 1)
for k in range(1, K + 1):
    mu_uniform[k] = 2 * S_uniform[k]
delta_uniform = compute_free_mult_defect(m_uniform, mu_uniform, K)
U_uniform = compute_cc_upper_bound(M_uniform)

if U_uniform < 0.05 * math.log2(N) * delta_uniform:
    conjecture_holds = False
    counterexample = f"Uniform matrix: U < 0.05 log2(N) delta: n={n}, U={U_uniform}, delta

# Build rank-1 matrix
M_rank1 = build_rank1_matrix(N)
Q_rank1 = matrix_mult(matrix_transpose(M_rank1), M_rank1)
m_rank1 = compute_moments(Q_rank1, K)
chi_rank1 = lagrange_inversion(m_rank1, K)
S_rank1 = compute_S_transform(chi_rank1, K)
mu_rank1 = [0.0] * (K + 1)
for k in range(1, K + 1):
    mu_rank1[k] = 2 * S_rank1[k]
delta_rank1 = compute_free_mult_defect(m_rank1, mu_rank1, K)
U_rank1 = compute_cc_upper_bound(M_rank1)

if U_rank1 < 0.05 * math.log2(N) * delta_rank1:
    conjecture_holds = False
    counterexample = f"Rank-1 matrix: U < 0.05 log2(N) delta: n={n}, U={U_rank1}, delta={d

# Build AND matrix
M_and = build_and_matrix(N)
Q_and = matrix_mult(matrix_transpose(M_and), M_and)
m_and = compute_moments(Q_and, K)
chi_and = lagrange_inversion(m_and, K)
S_and = compute_S_transform(chi_and, K)
mu_and = [0.0] * (K + 1)
for k in range(1, K + 1):
    mu_and[k] = 2 * S_and[k]
delta_and = compute_free_mult_defect(m_and, mu_and, K)
U_and = compute_cc_upper_bound(M_and)

if U_and < 0.05 * math.log2(N) * delta_and:
    conjecture_holds = False
    counterexample = f"AND matrix: U < 0.05 log2(N) delta: n={n}, U={U_and}, delta={delta_
```

```
        # Build identity matrix
        M_identity = build_identity_matrix(N)
        Q_identity = matrix_mult(matrix_transpose(M_identity), M_identity)
        m_identity = compute_moments(Q_identity, K)
        chi_identity = lagrange_inversion(m_identity, K)
        S_identity = compute_S_transform(chi_identity, K)
        mu_identity = [0.0] * (K + 1)
        for k in range(1, K + 1):
            mu_identity[k] = 2 * S_identity[k]
        delta_identity = compute_free_mult_defect(m_identity, mu_identity, K)
        U_identity = compute_cc_upper_bound(M_identity)

        if U_identity < 0.05 * math.log2(N) * delta_identity:
            conjecture_holds = False
            counterexample = f"Identity matrix: U < 0.05 log2(N) delta: n={n}, U={U_identity}, del

        metric_values.append(delta_uniform)

    return {
        "metric_name": "free_mult_defect",
        "metric_value": sum(metric_values) / len(metric_values),
        "instances_tested": len(n_values) * 4,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17,
    results = []
    for seed in seeds:
        result = run_trial(seed)
        result["seed"] = seed
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results]
    mean_metric = sum(metric_values) / len(metric_values)
    std_metric = math.sqrt(sum((x - mean_metric) ** 2 for x in metric_values) / len(metric_values)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.9:
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fr
    else:
        counterexamples = [r["counterexample"] for r in results if not r["conjecture_holds"]]
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample={counterexamples[0]} first_failing_seed={first_fa
```

6. Per-seed results

Seed	Metric value	Holds?	Counterexample
11	1.230422830748301	✓	
23	1.2395776684808344	✓	

Seed	Metric value	Holds?	Counterexample
37	1.2317113622119065	✗	Rank-1 matrix: U < 0.05 log2(N) delta: n=3, U=0.0, delta=0.7569444444444444
53	1.2490015016843579	✗	
71	1.2557235681809806	✗	
89	1.2226559649863677	✗	
103	1.2417354806168643	✗	
127	1.2319117424220483	✗	Identity matrix: U < 0.05 log2(N) delta: n=6, U=0.0, delta=0.75
149	1.2237111982190534	✗	
167	1.249338667605091	✗	Identity matrix: U < 0.05 log2(N) delta: n=6, U=0, delta=0.75
191	1.2310477174632115	✗	Identity matrix: U < 0.05 log2(N) delta: n=4, U=0.0, delta=0.75
211	1.222971697955392	✗	Identity matrix: U < 0.05 log2(N) delta: n=3, U=0.0, delta=0.75
233	1.2407496270329395	✗	Identity matrix: U < 0.05 log2(N) delta: n=4, U=0.0, delta=0.75
257	1.2584106427236494	✗	Identity matrix: U < 0.05 log2(N) delta: n=5, U=0.0, delta=0.75
277	1.2442368060210691	✗	
311	1.2416027413367843	✗	Identity matrix: U < 0.05 log2(N) delta: n=4, U=0.0, delta=0.75
347	1.2406850207204478	✗	
389	1.2176231754293128	✗	Identity matrix: U < 0.05 log2(N) delta: n=6, U=0.0, delta=0.75
421	1.2217622909689179	✗	
463	1.2462567547666548	✗	
503	1.2258976570350884	✗	
547	1.2174309166311343	✗	
593	1.218306819204241	✗	Identity matrix: U < 0.05 log2(N) delta: n=3, U=0.0, delta=0.75
631	1.2381510000250828	✗	Rank-1 matrix: U < 0.05 log2(N) delta: n=3, U=0.0, delta=0.9908707865168539

Seed	Metric value	Holds?	Counterexample
677	1.2508519087688272	✗	
727	1.242301935238181	✗	
773	1.2526578087928977	✗	Identity matrix: U < 0.05 log2(N) delta: n=5, U=0.0, delta=0.75
821	1.219413383084314	✗	Identity matrix: U < 0.05 log2(N) delta: n=5, U=0.0, delta=0.75
877	1.2409823554557478	✗	Identity matrix: U < 0.05 log2(N) delta: n=4, U=0.0, delta=0.75
929	1.215808549927829	✗	

Aggregate statistics:

Statistic	Value
n_seeds	30
metric_mean	1.2354312931245843
metric_std	0.012680527897105227
metric_ci95_half	0.004630274113545388
metric_min	1.215808549927829
metric_max	1.2584106427236494
support_fraction	0.5333333333333333

7. Test stdout (last 2KB)

```
16, "conjecture_holds": true, "counterexample": "", "seed": 503}
TRIAL: {"metric_name": "free_mult_defect", "metric_value": 1.2174309166311343, "instances_tested": 1
TRIAL: {"metric_name": "free_mult_defect", "metric_value": 1.218306819204241, "instances_tested": 16
TRIAL: {"metric_name": "free_mult_defect", "metric_value": 1.2381510000250828, "instances_tested": 1
1 matrix: U < 0.05 log2(N) delta: n=3, U=0.0, delta=0.9908707865168539", "seed": 631}
TRIAL: {"metric_name": "free_mult_defect", "metric_value": 1.2508519087688272, "instances_tested": 1
TRIAL: {"metric_name": "free_mult_defect", "metric_value": 1.242301935238181, "instances_tested": 16
TRIAL: {"metric_name": "free_mult_defect", "metric_value": 1.2526578087928977, "instances_tested": 1
TRIAL: {"metric_name": "free_mult_defect", "metric_value": 1.219413383084314, "instances_tested": 16
TRIAL: {"metric_name": "free_mult_defect", "metric_value": 1.2409823554557478, "instances_tested": 1
TRIAL: {"metric_name": "free_mult_defect", "metric_value": 1.215808549927829, "instances_tested": 16
RESULT: FALSIFIED counterexample=Rank-1 matrix: U < 0.05 log2(N) delta: n=3, U=0.0, delta=0.756944444
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> The per-seed data explicitly shows counterexamples: rank-1 and identity matrices have CC upper bound U=0 (constant communication, since DISJ-like structure is absent) yet δ ≈ 0.75, directly violating part (i) of the conjecture with c ≥ 1/64. The support_fraction is only 0.533 with CI [0.367, 0.733], meaning roughly half the seeded trials already refute the conjecture — this is not 'SUPPORTED'. The metric_mean ≈ 1.235 appears to be an unrelated aggregate quantity, not the conjecture's truth indic

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test's RESULT line reports FALSIFIED with explicit counterexamples (Identity and Rank-1 matrices at n=3,4,5 having U=0 but δ≈0.75–0.99), violating | next: Restrict the conjecture to Boolean matrices of high combinatorial rank (e.g., exclude low-rank/structured M) or replace U with a lower bound on CC_R rather than any certified upper bound.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	166994
2	preregistration	claude_max	opus	0	0	6503
3	novelty	claude_max	opus	0	0	3399
4	novelty	claude_max	opus	0	0	10103
5	test_gen	mistral	codestral-latest	0	0	318739
6	critic	claude_max	opus	0	0	10730
7	judge	claude_max	opus	0	0	5074

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 521542 ms total latency. Provider mix: {'claude_max': 6, 'mistral': 1}

(full prompt+response transcripts available in `research/audit/6c8daf2ab19d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6c8daf2ab19d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/6c8daf2ab19d.tar.gz` (if generated)